DATABASE AND COMPUTER NETWORKS

CIS095-1

Assignment 1

Aspire Fitness – Database Design

Student Name: Kripa Parajuli

Student ID: 2510112344

Section: B (L4 SEM2)

Submission Date: 13 March, 2026

Table of Contents

Introduction 1

Scenario Description 2

Assumptions 3

Task 1: Entity Relationship Model 4

- Entities and Their Attributes 4
- Relationships Between Entities 4
- Entity Relationship Diagram 6

Task 2: Physical Database Design 7

- Skeleton Tables (Normalized Relations) 7
- Data Dictionary 7
  - Table 1: Member 7
  - Table 2: FitnessStaff 8
  - Table 3: Room 8
  - Table 4: Class 9
  - Table 5: FacilityBooking 9
  - Table 6: ClassBooking 10

Task 3: Implementation of the Database 11

- Part A – Create Tables 11
  - Create Table: Member 11
  - Create Table: FitnessStaff 11
  - Create Table: Room 12
  - Create Table: Class 12
  - Create Table: FacilityBooking 13
  - Create Table: ClassBooking 14
- Part B – Insert Data 14
  - Insert Data: Member 14
  - Insert Data: FitnessStaff 15
  - Insert Data: Room 15
  - Insert Data: Class 16
  - Insert Data: FacilityBooking 16

Insert Data: ClassBooking 17
Task 4: Query Design and Implementation 18
Part A – List of Queries and Their Purpose 18
Part B – SQL Queries Implemented 18
Query 1: Full Member List 18
Query 2: Current Week Facility Bookings 19
Query 3: Classes by a Specific Instructor 20
Query 4: Classes in a Specific Room 20
Query 5: Member Booking Summary 21
Query 6: Class Bookings Count per Member 22
Query 7: Most Active Members This Month 23
Query 8: Members Over Free Class Limit 24
Query 9: Room Usage Summary 25
Query 10: Members with Extra Charges 26
Discussion and Critical Reflection 27
Conclusion 27
References 28
Appendix 29

Table of Figures

Figure 1: Entity Relationship Diagram for Aspire Fitness 6
Figure 2: Member table created 11
Figure 3: FitnessStaff table created 12
Figure 4: Room table created 12
Figure 5: Class table created 13
Figure 6: FacilityBooking table created 13
Figure 7: ClassBooking table created 14
Figure 8: Member table with inserted data 14
Figure 9: FitnessStaff table with inserted data 15
Figure 10: Room table with inserted data 15
Figure 11: Class table with inserted data 16
Figure 12: FacilityBooking table with inserted data 16
Figure 13: ClassBooking table with inserted data 17
Figure 14: Query 1 output 19
Figure 15: Query 2 output 20
Figure 16: Query 3 output 20
Figure 17: Query 4 output 21
Figure 18: Query 5 output 22
Figure 19: Query 6 output 23
Figure 20: Query 7 output 24
Figure 21: Query 8 output 25
Figure 22: Query 9 output 26
Figure 23: Query 10 output 26

Abstract

This report will involve my demonstration of how I developed and created a database in a sports club known as Aspire Fitness. The club had to find a way of managing room bookings and fitness class bookings among the club members. In order to accomplish this my entity relationship model was developed, the tables were arranged in an orderly manner and the database was constructed in MySQL. I also prepared ten SQL queries to demonstrate that the database does work the way that the club requires it to.

Introduction

This assignment involved me designing and developing a database to support a local sports club by the name Aspire Fitness. The assignment entailed determining the business needs, designing the ERD, normalization of tables, and implementing the database in MySQL.

I have followed four key steps: I created the ERD, I wrote skeleton tables and a data dictionary, I wrote SQL to create tables and insert test data and I used the query to demonstrate how the database is used in supporting the daily operations of the club.

Even though this was the first formal database unit that I took, I was already familiar with the basics of databases such as primary keys, foreign keys, and SQL. Thanks to this assignment I came to realize how to put data together and transform business needs into a working database better than before.

Scenario Description

Aspire Fitness is a sports and fitness club, having three rooms that are bookable: the Main Hall, Yoga Hall, and Small Dance Room. The club consists of approximately 20 employees (trainers, instructors, and administrative personnel).

The database is used to take care of two major services whereby, it is used to book rooms and also book classes. The members will be able to book a single room weekly to a maximum of two hours and no bookings may occur in conflict with other bookings or classes. The members are also allowed to have up to five classes as a week at no cost and an extra class is at a cost of Rs 500.

Members are given a membership number and all their personal information is contained in the system. Information of the staff is also documented. The fee is paid by members on weekly basis, which is Rs 1,000, and an additional amount of Rs 500 is paid by the members whenever they attend an extra class after five free classes.

Assumptions

When coming up with the database design, the scenario did not clearly elaborate on some of the details. Thus, the assumptions that were made to assist in structuring the tables and relationships were as follows.

1. Registration of Members - Before members could make reservations of rooms or classes, they have to be registered in the system.
2. Room Booking Limit - There is the room booking limit, each member can book one room per week for a maximum of two hours.
3. Class Schedule - The classes are fixed in terms of their room, day, time, and the instructor.
4. Limit to Class Booking - The members are allowed to attend unlimited classes however the initial five classes are free per week. Each extra class costs Rs 500.
5. Medical Information - Field of medical conditions is an optional field to the members.
6. Staff Roles - Every staff does not teach classes; some members of the staff are administrative members and are not associated with classes.
7. Weekly Fee Calculation- The weekly fee will be determined depending on the amount of additional classes booked.
8. Validation of the Bookings - The bookings made are not automatically validated in the database and therefore the staff is required to verify the availability of a room before booking one.

Task 1: Entity Relationship Model

Entity Relationship Model (ERM) is a diagram employed to design the information that a system must store, and how various data portions relate. The entities, which are what we would like to store information about, are represented by each box in the diagram. The lines between the boxes show the relationships between the entities.

Entities and Their Attributes

Based on the reading of the Aspire Fitness scenario, there were six entities within which the database ought to carry information:

Entity	Information Stored
Member	Stores includes all personal information of its members such as their unique membership number, name, address, date of birth, phone number, email, and medical conditions.
FitnessStaff	Stores unique staff number, name, job position and phone number of each member.
Room	Stores the room ID, room name and the maximum capacity of the rooms in the room.
Class	Stores the respective unique class code, class name, instructor assigned, room, day, start time and maximum number of members of a specific class.
FacilityBooking	Records all room bookings, such as the booking ID, member, room, date, start and end time and the intended activity.
ClassBooking	Records all the booking of classes with booking ID, member, class code, the date of booking and the presence of an additional charge.

Relationships Between Entities

After obtaining all the entities, I then tried to figure out how each one is connected to the others. Such links are referred to as relationships. Each relationship and cardinality have been explained below and this is the number of records that can be connected to each cardinality.

MEMBER ↔ FacilityBooking:

- A member is able to make numerous facility bookings in the long run.
- However, each facility booking always belongs to only one member.
- Cardinality: One-to-Many (1:M)

MEMBER ↔ ClassBooking:

- A member has the ability to make numerous reservations of classes.
- However, each class booking is always linked to only one member.
- Cardinality: One-to-Many (1:M)

ROOM ↔ FacilityBooking:

- A room can be presented in a number of facility bookings since a room can be booked by various members on separate occasions.
- However, every single booking will always refer to one room.
- Cardinality: One-to-Many (1:M)

ROOM ↔ Class:

- During the week, there are numerous classes that can be conducted in a room.
- However, in each instance the classes are always confined in only one room.
- Cardinality: One-to-Many (1:M)

FitnessStaff ↔ Class:

- An instructor as a staff member may teach a great number of classes.
- However, one instructor is assigned to each class.
- Cardinality: One-to-Many (1:M)

Class ↔ ClassBooking:

- There may be a large number of bookings by various members of a class.
- Nevertheless, there is always a particular class when each individual class booking occurs.
- Cardinality: One-to-Many (1:M)

Entity Relationship Diagram

Visual paradigm was used to draw the ERD below. It depicts all the six entities, their properties and the way they are related to one another. The marks on the ends of the lines indicate the kind of the relationship between each entity.

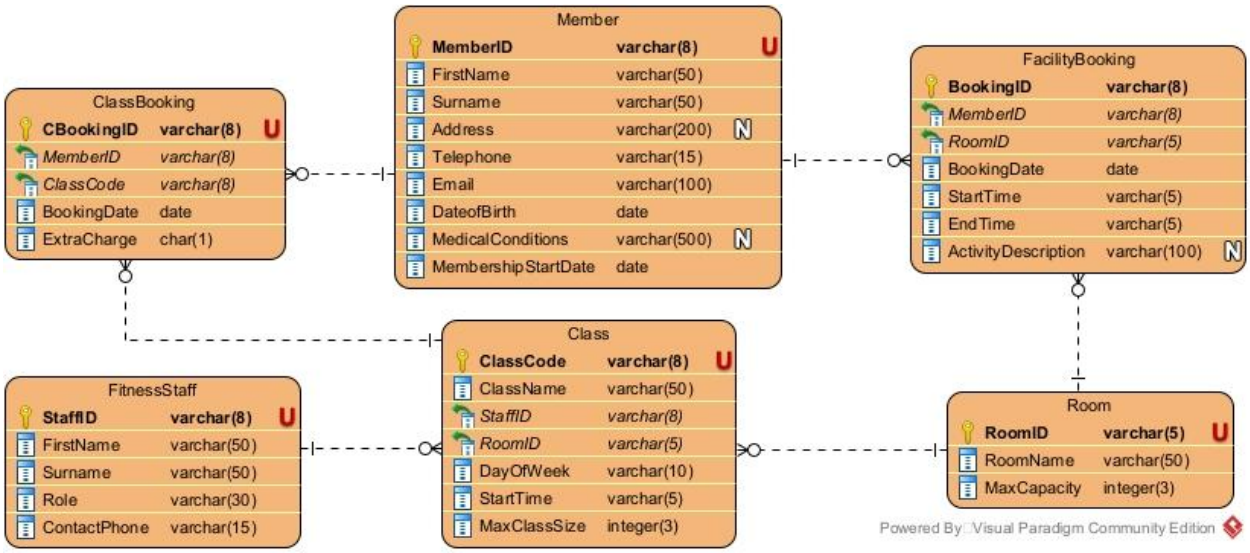

Figure 1: Entity Relationship Diagram for Aspire Fitness

Task 2: Physical Database Design

In this section, the way the database was planned prior to writing a single SQL code is explained. The reason that I employed MySQL in this project is because it is free and is widely applied in educational and industrial sectors. MySQL Workbench was also used in writing SQL queries and seeing results.

Skeleton Tables (Normalized Relations)

The main tables and their columns are displayed in skeleton tables. The primary key is used to uniquely identify the records and the foreign keys to relate two different tables. The tables in the database are the following:

- Member (MemberID, FirstName, Surname, Address, Telephone, Email, DateOfBirth, MedicalConditions, MembershipStartDate)
- FitnessStaff (StaffID, FirstName, Surname, Role, ContactPhone)
- Room (RoomID, RoomName, MaxCapacity)
- Class (ClassCode, ClassName, StaffID*, RoomID*, DayOfWeek, StartTime, MaxClassSize)
- FacilityBooking (BookingID, MemberID*, RoomID*, BookingDate, StartTime, EndTime, ActivityDescription)
- ClassBooking (CBookingID, MemberID*, ClassCode*, BookingDate, ExtraCharge)

The database tables were brought to the Third Normal Form (3NF) so that data duplication can be minimized and that the data can be ordered. As an example, the information about instructors is stored in FitnessStaff table rather than in class tables.

Data Dictionary

The data dictionary specifies the format of each table like field names, data type and other information used in every column of the database.

Table 1: Member

Field Name	Data Type	Length	Index	Null?	Validation	Description
MemberID	VARCHAR	8	PK	No		Unique identifier for each member
FirstName	VARCHAR	50		No		Member's first name

Surname	VARCHAR	50		No		Member's surname
Address	VARCHAR	200		No		Full postal address
Telephone	VARCHAR	15		No		Contact phone number
Email	VARCHAR	100		No	Must contain @	Email address
DateOfBirth	DATE			No	Must be past date	Member's date of birth
MedicalConditions	VARCHAR	500		Yes		Any declared medical conditions (optional)
MembershipStartDate	DATE			No		Date membership began

Table 2: FitnessStaff

Field Name	Data Type	Length	Index	Null?	Validation	Description
StaffID	VARCHAR	8	PK	No		Unique identifier for each staff member
FirstName	VARCHAR	50		No		Staff first name
Surname	VARCHAR	50		No		Staff surname
Role	VARCHAR	30		No	Personal Trainer, Class Instructor, Gym Staff, Manager, Administration	Staff role
ContactPhone	VARCHAR	15		No		Staff contact number

Table 3: Room

Field Name	Data Type	Length	Index	Null?	Validation	Description
RoomID	VARCHAR	5	PK	No		Unique room identifier

RoomName	VARCHAR	50		No	Main Hall, Yoga Hall, Small Dance Room	Name/type of the room
MaxCapacity	INT	3		No	Must be > 0	Maximum number of people the room can hold

Table 4: Class

Field Name	Data Type	Length	Index	Null?	Validation	Description
ClassCode	VARCHAR	8	PK	No		Unique identifier for each class
ClassName	VARCHAR	50		No		Name of the class (e.g. Yoga, Zumba)
StaffID	VARCHAR	8	FK	No		References FitnessStaff (StaffID)
RoomID	VARCHAR	5	FK	No		References Room (RoomID)
DayOfWeek	VARCHAR	10		No	Monday.. ...Sunday	Day the class runs
StartTime	VARCHAR	5		No	HH:MM format	Scheduled start time
MaxClassSize	INT	3		No	Must be > 0	Maximum number of members who can book

Table 5: FacilityBooking

Field Name	Data Type	Length	Index	Null?	Validation	Description
BookingID	VARCHAR	8	PK	No		Unique booking reference
MemberID	VARCHAR	8	FK	No		References Member (MemberID)

RoomID	VARCHAR	5	FK	No		References Room (RoomID)
BookingDate	DATE			No	Cannot be past date	Date of the booking
StartTime	VARCHAR	5		No	HH:MM format	Start of the booked time slot
EndTime	VARCHAR	5		No	Max 2 hrs. after StartTime	End of the booked time slot
ActivityDescription	VARCHAR	100		Yes		Activity planned (e.g. Volleyball, Basketball)

Table 6: ClassBooking

Field Name	Data Type	Length	Index	Null?	Validation	Description
CBookingID	VARCHAR	8	PK	No		Unique class booking reference
MemberID	VARCHAR	8	FK	No		References Member (MemberID)
ClassCode	VARCHAR	8	FK	No		References Class (ClassCode)
BookingDate	DATE			No		Date the booking was made
ExtraCharge	CHAR	1		No	Y or N	Whether an extra Rs 500 charge applies

Task 3: Implementation of the Database

The SQL code that was used to create the Aspire Fitness database on MySQL is recorded. Part A discusses the CREATE TABLE that was used to create each table. Part B explains the INSERTs that will be used to insert sample data into each table in order to test that everything is working.

Part A – Create Tables

Create Table: Member

This forms the Member table whose primary key is the MemberID.

```sql
CREATE DATABASE AspireFitness;
USE AspireFitness;

CREATE TABLE Member (
  MemberID VARCHAR(8) NOT NULL,
  FirstName VARCHAR(50) NOT NULL,
  Surname VARCHAR(50) NOT NULL,
  Address VARCHAR(200) NOT NULL,
  Telephone VARCHAR(15) NOT NULL,
  Email VARCHAR(100) NOT NULL,
  DateOfBirth DATE NOT NULL,
  MedicalConditions VARCHAR(500),
  MembershipStartDate DATE NOT NULL,
  PRIMARY KEY (MemberID)
);
```

1	13:50:01	CREATE DATABASE AspireFitness	1 row(s) affected	0.078 sec
2	13:50:05	USE AspireFitness	0 row(s) affected	0.016 sec
3	13:50:09	CREATE TABLE Member (MemberID VARCHAR(8) NOT NULL, FirstName VARCHAR(50) NOT NULL,...	0 row(s) affected	0.109 sec

Figure 2: Member table created

Create Table: FitnessStaff

This forms the FitnessStaff table where StaffID is the primary key.

```sql
CREATE TABLE FitnessStaff (
  StaffID VARCHAR(8) NOT NULL,
  FirstName VARCHAR(50) NOT NULL,
  Surname VARCHAR(50) NOT NULL,
  Role VARCHAR(30) NOT NULL,
  ContactPhone VARCHAR(15) NOT NULL,
  PRIMARY KEY (StaffID)
);
```

4 13:50:14 CREATE TABLE FitnessStaff (StaffID VARCHAR(8) NOT NULL, FirstName VARCHAR(50) NOT NULL,... 0 row(s) affected 0.109 sec

Figure 3: FitnessStaff table created

Create Table: Room

This will generate the Room table whose primary key will be RoomID.

```sql
CREATE TABLE Room (
  RoomID VARCHAR(5) NOT NULL,
  RoomName VARCHAR(50) NOT NULL,
  MaxCapacity INT NOT NULL,
  PRIMARY KEY (RoomID)
);
```

5 13:50:18 CREATE TABLE Room (RoomID VARCHAR(5) NOT NULL, RoomName VARCHAR(50) NOT NULL, ... 0 row(s) affected 0.109 sec

Figure 4: Room table created

Create Table: Class

This is to generate the Class table with ClassCode as the primary key. The FitnessStaff and Room tables have the foreign keys known as StaffID and RoomID.

```sql
CREATE TABLE Class (
  ClassCode VARCHAR(8) NOT NULL,
  ClassName VARCHAR(50) NOT NULL,
  StaffID VARCHAR(8) NOT NULL,
  RoomID VARCHAR(5) NOT NULL,
  DayOfWeek VARCHAR(10) NOT NULL,
  StartTime VARCHAR(5) NOT NULL,
  MaxClassSize INT NOT NULL,
  PRIMARY KEY (ClassCode),
  FOREIGN KEY (StaffID) REFERENCES FitnessStaff(StaffID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

```
6  13:50:23  CREATE TABLE Class (  ClassCode VARCHAR(8) NOT NULL,  ClassName VARCHAR(50) NOT NULL, ...  0 row(s) affected  0.093 sec
```

Figure 5: Class table created

Create Table: FacilityBooking

This forms the table FacilityBooking where the primary key is BookingID. The MemberID and RoomID are foreign keys that are connected to the Member and Room tables.

```sql
CREATE TABLE FacilityBooking (
  BookingID VARCHAR(8) NOT NULL,
  MemberID VARCHAR(8) NOT NULL,
  RoomID VARCHAR(5) NOT NULL,
  BookingDate DATE NOT NULL,
  StartTime VARCHAR(5) NOT NULL,
  EndTime VARCHAR(5) NOT NULL,
  ActivityDescription VARCHAR(100),
  PRIMARY KEY (BookingID),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

```
7  13:50:27  CREATE TABLE FacilityBooking (  BookingID VARCHAR(8) NOT NULL,  MemberID VARCHAR(8) NOT ...  0 row(s) affected  0.125 sec
```

Figure 6: FacilityBooking table created

Create Table: ClassBooking

This forms the ClassBooking table with CBookingID as the primary key. MemberID and ClassCode are foreign keys which are connected to the Member and Class table.

```
CREATE TABLE ClassBooking (
    CBookingID VARCHAR(8) NOT NULL,
    MemberID VARCHAR(8) NOT NULL,
    ClassCode VARCHAR(8) NOT NULL,
    BookingDate DATE NOT NULL,
    ExtraCharge CHAR(1) NOT NULL,
    PRIMARY KEY (CBookingID),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (ClassCode) REFERENCES Class(ClassCode)
);
```

8 13:50:32 CREATE TABLE ClassBooking (CBookingID VARCHAR(8) NOT NULL, MemberID VARCHAR(8) NOT ... 0 row(s) affected 0.140 sec

Figure 7: ClassBooking table created

Part B – Insert Data

Insert Data: Member

```
INSERT INTO Member VALUES
('110','Kripa','Parajuli','Dadhikot, Bhaktapur','9001234567','kripa.parajuli@email.com','2006-09-18',NULL,'2024-01-10'),
('120','Bibek','Bharati','Dolalghat, Kavrepalanchowk','97711234002','bibek.bharati@email.com','2006-12-22',NULL,'2024-02-01'),
('130','Soniya','Gurung','Balkot, Bhaktapur','97711234003','soniya.gurung@email.com','2007-11-02','Asthma','2023-09-15'),
('140','Aananya','Shrestha','Patan, Lalitpur','97711234004','aananya.shrestha@email.com','2000-03-30','Knee injury','2024-03-01');
```

MemberID	FirstName	Surname	Address	Telephone	Email	DateOfBirth	MedicalConditions	MembershipStartDate
110	Kripa	Parajuli	Dadhikot, Bhaktapur	9001234567	kripa.parajuli@email.com	2006-09-18	NULL	2024-01-10
120	Bibek	Bharati	Dolalghat, Kavrepalanchowk	97711234002	bibek.bharati@email.com	2006-12-22	NULL	2024-02-01
130	Soniya	Gurung	Balkot, Bhaktapur	97711234003	soniya.gurung@email.com	2007-11-02	Asthma	2023-09-15
140	Aananya	Shrestha	Patan, Lalitpur	97711234004	aananya.shrestha@email.com	2000-03-30	Knee injury	2024-03-01
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

9 13:50:40 INSERT INTO Member VALUES ('110','Kripa','Parajuli','Dadhikot, Bhaktapur','9001234567','kripa.parajuli@... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 8: Member table with inserted data

Insert Data: FitnessStaff

```
INSERT INTO FitnessStaff VALUES
('S01','Ramesh','Shrestha','Class Instructor','9841234501'),
('S02','Suman','Tamang','Personal Trainer','9841234502'),
('S03','Anita','Maharjan','Class Instructor','9841234503'),
('S04','Bikash','Karki','Manager','9841234504');
```

StaffID	FirstName	Surname	Role	ContactPhone
S01	Ramesh	Shrestha	Class Instructor	9841234501
S02	Suman	Tamang	Personal Trainer	9841234502
S03	Anita	Maharjan	Class Instructor	9841234503
S04	Bikash	Karki	Manager	9841234504
NULL	NULL	NULL	NULL	NULL

10 13:50:44 INSERT INTO FitnessStaff VALUES ('S01','Ramesh','Shrestha','Class Instructor','9841234501'), ('S02','Sum... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.078 sec

Figure 9: FitnessStaff table with inserted data

Insert Data: Room

```
INSERT INTO Room VALUES
('R001','Main Hall',80),
('R002','Yoga Hall',25),
('R003','Small Dance Room',10);
```

RoomID	RoomName	MaxCapacity
R001	Main Hall	80
R002	Yoga Hall	25
R003	Small Dance Room	10
NULL	NULL	NULL

12 13:50:53 INSERT INTO Class VALUES ('CLS001','Yoga Flow','S01','R002','Monday','09:00',20), ('CLS002','Zumba','... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 10: Room table with inserted data

Insert Data: Class

```
INSERT INTO Class VALUES
('CLS001','Yoga Flow','S01','R002','Monday','09:00',20),
('CLS002','Zumba','S03','R001','Wednesday','18:30',40),
('CLS003','Pilates','S01','R003','Friday','10:00',10),
('CLS004','Tai Chi','S03','R002','Tuesday','08:00',15);
```

ClassCode	ClassName	StaffID	RoomID	DayOfWeek	StartTime	MaxClassSize
CLS001	Yoga Flow	S01	R002	Monday	09:00	20
CLS002	Zumba	S03	R001	Wednesday	18:30	40
CLS003	Pilates	S01	R003	Friday	10:00	10
CLS004	Tai Chi	S03	R002	Tuesday	08:00	15
NULL	NULL	NULL	NULL	NULL	NULL	NULL

12 13:50:53 INSERT INTO Class VALUES ('CLS001','Yoga Flow','S01','R002','Monday','09:00',20), ('CLS002','Zumba','... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 11: Class table with inserted data

Insert Data: FacilityBooking

```
INSERT INTO FacilityBooking VALUES
('FB001','110','R001','2026-03-10','10:00','12:00','Volleyball'),
('FB002','120','R002','2026-03-11','14:00','16:00','Yoga practice'),
('FB003','130','R001','2026-03-12','09:00','11:00','Basketball'),
('FB004','140','R003','2026-03-13','13:00','15:00','Dance practice');
```

BookingID	MemberID	RoomID	BookingDate	StartTime	EndTime	ActivityDescription
FB001	110	R001	2026-03-10	10:00	12:00	Volleyball
FB002	120	R002	2026-03-11	14:00	16:00	Yoga practice
FB003	130	R001	2026-03-12	09:00	11:00	Basketball
FB004	140	R003	2026-03-13	13:00	15:00	Dance practice
NULL	NULL	NULL	NULL	NULL	NULL	NULL

13 13:50:57 INSERT INTO FacilityBooking VALUES ('FB001','110','R001','2026-03-10','10:00','12:00','Volleyball'), ('FB0... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 12: FacilityBooking table with inserted data

Insert Data: ClassBooking

```sql
INSERT INTO ClassBooking VALUES
('CB001','110','CLS001','2026-03-05','N'),
('CB002','110','CLS002','2026-03-05','N'),
('CB003','120','CLS003','2026-03-06','N'),
('CB004','130','CLS001','2026-03-06','N'),
('CB005','130','CLS004','2026-03-07','N'),
('CB006','130','CLS002','2026-03-07','Y'),
('CB007','130','CLS001','2026-03-08','N'),
('CB008','130','CLS002','2026-03-08','N'),
('CB009','130','CLS003','2026-03-09','Y');
```

CBookingID	MemberID	ClassCode	BookingDate	ExtraCharge
CB001	110	CLS001	2026-03-05	N
CB002	110	CLS002	2026-03-05	N
CB003	120	CLS003	2026-03-06	N
CB004	130	CLS001	2026-03-06	N
CB005	130	CLS004	2026-03-07	N
CB006	130	CLS002	2026-03-07	Y
CB007	130	CLS001	2026-03-08	N
CB008	130	CLS002	2026-03-08	N
CB009	130	CLS003	2026-03-09	Y
NULL	NULL	NULL	NULL	NULL

14 13:51:01 INSERT INTO ClassBooking VALUES ('CB001','110','CLS001','2026-03-05','N'), ('CB002','110','CLS002','2... 9 row(s) affected Records: 9 Duplicates: 0 Warnings: 0 0.063 sec

Figure 13: ClassBooking table with inserted data

Task 4: Query Design and Implementation

This section discusses the SQL queries that were developed in the Aspire Fitness database. The queries are applied to get the useful information in the database. Part A gives a description of what each query will do and B contains the SQL code and a brief explanation.

Part A – List of Queries and Their Purpose

#	Query Name	Justification
1	Full Member List	lists all registered members.
2	Current Week Facility Bookings	Displays room bookings of the current week.
3	Classes by a Specific Instructor	Lists classes taught by a particular instructor.
4	Classes in a Specific Room	Displays classes that will take place in a given room.
5	Member Booking Summary	Shows the booking of a member by week and room.
6	Class Bookings Count per Member	This counts the number of classes that member booked this week.
7	Most Active Members This Month	Displays the members that made the most bookings during this month.
8	Members Over Free Class Limit	This is the target of members who made over five bookings in one week.
9	Room Usage Summary	Displays the frequency of room bookings.
10	Members with Extra Charges	Are members who are required to take extra classes.

Part B – SQL Queries Implemented

Query 1: Full Member List

Detail	Description
Purpose	Shows a list of all registered members ordered by surname.
Explanation	SELECT is used to pick the columns we want to show from the Member table. ORDER BY Surname sorts the results from A to Z by surname.

```
SELECT MemberID, FirstName, Surname, Telephone, Email
FROM Member
ORDER BY Surname;
```

MemberID	FirstName	Surname	Telephone	Email
120	Bibek	Bharati	97711234002	bibek.bharati@email.com
130	Soniya	Gurung	97711234003	soniya.gurung@email.com
110	Kripa	Parajuli	9001234567	kripa.parajuli@email.com
140	Aananya	Shrestha	97711234004	aananya.shrestha@email.com
NULL	NULL	NULL	NULL	NULL

Figure 14: Query 1 output

Query 2: Current Week Facility Bookings

Detail	Description
Purpose	Shows all room bookings made in the current week.
Explanation	JOIN connects the FacilityBooking, Member and Room tables so we can see the member's name and room name together. WHERE filters the results to only show bookings from the current week.

```
SELECT FacilityBooking.BookingID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Room.RoomName,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       FacilityBooking.EndTime,
       FacilityBooking.ActivityDescription
FROM FacilityBooking
JOIN Member ON FacilityBooking.MemberID = Member.MemberID
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE MONTH(FacilityBooking.BookingDate) = MONTH(CURDATE())
AND YEAR(FacilityBooking.BookingDate) = YEAR(CURDATE())
ORDER BY FacilityBooking.BookingDate, FacilityBooking.StartTime;
```

	BookingID	MemberName	RoomName	BookingDate	StartTime	EndTime	ActivityDescription
▶	FB001	Kripa Parajuli	Main Hall	2026-03-10	10:00	12:00	Volleyball
	FB002	Bibek Bharati	Yoga Hall	2026-03-11	14:00	16:00	Yoga practice
	FB003	Soniya Gurung	Main Hall	2026-03-12	09:00	11:00	Basketball
	FB004	Aananya Shrestha	Small Dance Room	2026-03-13	13:00	15:00	Dance practice

Figure 15: Query 2 output

Query 3: Classes by a Specific Instructor

Detail	Description
Purpose	Shows all classes taught by a specific instructor.
Explanation	JOIN connects the Class, FitnessStaff and Room tables. WHERE filters by the instructor's StaffID. To search for a different instructor, just change the StaffID value in the WHERE clause.

```
SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       Room.RoomName,
       Class.MaxClassSize,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Class.StaffID = 'S01'
ORDER BY Class.DayOfWeek, Class.StartTime;
```

	ClassCode	ClassName	DayOfWeek	StartTime	RoomName	MaxClassSize	InstructorName
▶	CLS003	Pilates	Friday	10:00	Small Dance Room	10	Ramesh Shrestha
	CLS001	Yoga Flow	Monday	09:00	Yoga Hall	20	Ramesh Shrestha

Figure 16: Query 3 output

Query 4: Classes in a Specific Room

Detail	Description
Purpose	Shows all classes scheduled in a specific room.

Explanation	JOIN connects the Class, FitnessStaff and Room tables. WHERE filters by RoomID. ORDER BY sorts the results by day and time so the timetable is easy to read.

```sql
SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Room.RoomID = 'R002'
ORDER BY Class.DayOfWeek, Class.StartTime;
```

	ClassCode	ClassName	DayOfWeek	StartTime	InstructorName
▶	CLS001	Yoga Flow	Monday	09:00	Ramesh Shrestha
	CLS004	Tai Chi	Tuesday	08:00	Anita Maharjan

Figure 17: Query 4 output

Query 5: Member Booking Summary

Detail	Description
Purpose	Shows all room and class bookings for a specific member in the current week.
Explanation	UNION ALL combines results from FacilityBooking and ClassBooking into one list. A label column called BookingType is added so we can tell which type of booking each row is. WHERE filters by member ID and current week.

```sql
SELECT 'Facility Booking' AS BookingType,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       Room.RoomName,
       FacilityBooking.ActivityDescription AS Detail
FROM FacilityBooking
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE FacilityBooking.MemberID = '110'
UNION ALL
SELECT 'Class Booking' AS BookingType,
       ClassBooking.BookingDate,
       Class.StartTime,
       Room.RoomName,
       Class.ClassName AS Detail
FROM ClassBooking
JOIN Class ON ClassBooking.ClassCode = Class.ClassCode
JOIN Room ON Class.RoomID = Room.RoomID
WHERE ClassBooking.MemberID = '110'
ORDER BY BookingDate, StartTime;
```

BookingType	BookingDate	StartTime	RoomName	Detail
Class Booking	2026-03-05	09:00	Yoga Hall	Yoga Flow
Class Booking	2026-03-05	18:30	Main Hall	Zumba
Facility Booking	2026-03-10	10:00	Main Hall	Volleyball

Figure 18: Query 5 output

Query 6: Class Bookings Count per Member

Detail	Description
Purpose	Counts how many classes each member has booked this week.
Explanation	GROUP BY groups the results by member so we can count their bookings separately. COUNT counts the number of bookings for each member. ORDER BY sorts so the member with the most bookings appears at the top.

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
ORDER BY TotalClassBookings DESC;
```

MemberID	MemberName	TotalClassBookings
130	Soniya Gurung	6
110	Kripa Parajuli	2
120	Bibek Bharati	1

Figure 19: Query 6 output

Query 7: Most Active Members This Month

Detail	Description
Purpose	Shows which members have made the most bookings this month by adding up their room and class bookings.
Explanation	Two subqueries count room bookings and class bookings separately for each member. LEFT JOIN combines both counts into one result. IFNULL makes sure that if a member has no bookings, it shows 0 instead of blank. Results are sorted by total activity from highest to lowest.

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       IFNULL(FacilityCount.Total, 0) AS FacilityBookings,
       IFNULL(ClassCount.Total, 0) AS ClassBookings,
       IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) AS TotalActivity
FROM Member
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM FacilityBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) FacilityCount ON Member.MemberID = FacilityCount.MemberID
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM ClassBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) ClassCount ON Member.MemberID = ClassCount.MemberID
WHERE IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) > 0
ORDER BY TotalActivity DESC;
```

MemberID	MemberName	FacilityBookings	ClassBookings	TotalActivity
130	Soniya Gurung	1	6	7
110	Kripa Parajuli	1	2	3
120	Bibek Bharati	1	1	2
140	Aananya Shrestha	1	0	1

Figure 20: Query 7 output

Query 8: Members Over Free Class Limit

Detail	Description
Purpose	Shows members who have booked more than five classes this week and calculates their extra charge.
Explanation	GROUP BY groups the bookings by member. HAVING filters to only show members with more than five class bookings. The extra charge is calculated by multiplying the number of extra classes by 500.

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings,
       (COUNT(*) - 5) * 500 AS ExtraChargeRs
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
HAVING COUNT(*) > 5
ORDER BY TotalClassBookings DESC;
```

	MemberID	MemberName	TotalClassBookings	ExtraChargeRs
▶	130	Soniya Gurung	6	500

Figure 21: Query 8 output

Query 9: Room Usage Summary

Detail	Description
Purpose	Shows how many bookings each room has received in total.
Explanation	LEFT JOIN connects Room and FacilityBooking so even rooms with no bookings still appear. GROUP BY groups by room. COUNT counts the bookings for each room. Results are sorted from most booked to least booked.

```sql
SELECT Room.RoomID,
       Room.RoomName,
       Room.MaxCapacity,
       COUNT(FacilityBooking.BookingID) AS TotalBookings
FROM Room
LEFT JOIN FacilityBooking ON Room.RoomID = FacilityBooking.RoomID
GROUP BY Room.RoomID, Room.RoomName, Room.MaxCapacity
ORDER BY TotalBookings DESC;
```

	RoomID	RoomName	MaxCapacity	TotalBookings
▶	R001	Main Hall	80	2
	R002	Yoga Hall	25	1
	R003	Small Dance Room	10	1

Figure 22: Query 9 output

Query 10: Members with Extra Charges

Detail	Description
Purpose	Shows all members who have at least one extra charge class booking this week.
Explanation	WHERE filters records where ExtraCharge is Y and the booking is within the current week. JOIN connects to the Member table to show the member's name. DISTINCT makes sure each member only appears once even if they have multiple extra charge bookings.

```
SELECT DISTINCT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Member.Email
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
WHERE ClassBooking.ExtraCharge = 'Y'
ORDER BY Member.Surname;
```

	MemberID	Surname	MemberName	Email
▶	130	Gurung	Soniya Gurung	soniya.gurung@email.com

Figure 23: Query 10 output

Discussion and Critical Reflection

Overall, this was a successful project and I was in a position to accomplish the four activities. One of the difficulties was the determination of how to chart out the data into tables. I created two separate booking tables because combining them would have left many columns empty that was not good design. I also got to know about normalization- arranging data so as not to repeat. To illustrate, maintaining a separate Class table would imply it would only require updating one record in case an instructor is changed instead of multiple. Composing queries enabled me to know the distinction between WHERE filters before grouping and HAVING filters after grouping. The initial efforts in planning the database structure were very useful in saving time in implementation. I would also include a table on payments in the future and automate booking rules with triggers.

Conclusion

This project encompassed the entire work of creating a database design of the Aspire Fitness including scenario analysis and ERD development, construction of tables in MySQL and SQL queries to test the system. It consolidated my knowledge on database design, normalization and SQL, which gives me a major basis of learning in the future.

References

Oracle Corporation (2024) MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/ [Accessed: March 2026].

Connolly, T. and Begg, C. (2014) Database Systems: A Practical Approach to Design, Implementation, and Management. 6th edn. Harlow: Pearson Education.

Date, C.J. (2003) An Introduction to Database Systems. 8th edn. Boston: Addison-Wesley.

Appendix

```sql
CREATE DATABASE AspireFitness;
USE AspireFitness;

CREATE TABLE Member (
    MemberID VARCHAR(8) NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    Surname VARCHAR(50) NOT NULL,
    Address VARCHAR(200) NOT NULL,
    Telephone VARCHAR(15) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    DateOfBirth DATE NOT NULL,
    MedicalConditions VARCHAR(500),
    MembershipStartDate DATE NOT NULL,
    PRIMARY KEY (MemberID)
);

CREATE TABLE FitnessStaff (
    StaffID VARCHAR(8) NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    Surname VARCHAR(50) NOT NULL,
    Role VARCHAR(30) NOT NULL,
    ContactPhone VARCHAR(15) NOT NULL,
    PRIMARY KEY (StaffID)
);
```

```sql

CREATE TABLE Room (
    RoomID VARCHAR(5) NOT NULL,
    RoomName VARCHAR(50) NOT NULL,
    MaxCapacity INT NOT NULL,
    PRIMARY KEY (RoomID)
);

CREATE TABLE Class (
    ClassCode VARCHAR(8) NOT NULL,
    ClassName VARCHAR(50) NOT NULL,
    StaffID VARCHAR(8) NOT NULL,
    RoomID VARCHAR(5) NOT NULL,
    DayOfWeek VARCHAR(10) NOT NULL,
    StartTime VARCHAR(5) NOT NULL,
    MaxClassSize INT NOT NULL,
    PRIMARY KEY (ClassCode),
    FOREIGN KEY (StaffID) REFERENCES FitnessStaff(StaffID),
    FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

```sql
CREATE TABLE FacilityBooking (
  BookingID VARCHAR(8) NOT NULL,
  MemberID VARCHAR(8) NOT NULL,
  RoomID VARCHAR(5) NOT NULL,
  BookingDate DATE NOT NULL,
  StartTime VARCHAR(5) NOT NULL,
  EndTime VARCHAR(5) NOT NULL,
  ActivityDescription VARCHAR(100),
  PRIMARY KEY (BookingID),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);

CREATE TABLE ClassBooking (
  CBookingID VARCHAR(8) NOT NULL,
  MemberID VARCHAR(8) NOT NULL,
  ClassCode VARCHAR(8) NOT NULL,
  BookingDate DATE NOT NULL,
  ExtraCharge CHAR(1) NOT NULL,
  PRIMARY KEY (CBookingID),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (ClassCode) REFERENCES Class(ClassCode)
);
```

```sql
INSERT INTO Member VALUES
('110','Kripa','Parajuli','Dadhikot, Bhaktapur','9001234567','kripa.parajuli@email.com','2006-09-18',NULL,'2024-01-10'),
('120','Bibek','Bharati','Dolalghat, Kavrepalanchowk','97711234002','bibek.bharati@email.com','2006-12-22',NULL,'2024-02-01'),
('130','Soniya','Gurung','Balkot, Bhaktapur','97711234003','soniya.gurung@email.com','2007-11-02','Asthma','2023-09-15'),
('140','Aananya','Shrestha','Patan, Lalitpur','97711234004','aananya.shrestha@email.com','2000-03-30','Knee injury','2024-03-01');

INSERT INTO FitnessStaff VALUES
('S01','Ramesh','Shrestha','Class Instructor','9841234501'),
('S02','Suman','Tamang','Personal Trainer','9841234502'),
('S03','Anita','Maharjan','Class Instructor','9841234503'),
('S04','Bikash','Karki','Manager','9841234504');

INSERT INTO Room VALUES
('R001','Main Hall',80),
('R002','Yoga Hall',25),
('R003','Small Dance Room',10);

INSERT INTO Class VALUES
('CLS001','Yoga Flow','S01','R002','Monday','09:00',20),
('CLS002','Zumba','S03','R001','Wednesday','18:30',40),
('CLS003','Pilates','S01','R003','Friday','10:00',10),
('CLS004','Tai Chi','S03','R002','Tuesday','08:00',15);
```

```sql
INSERT INTO FacilityBooking VALUES
('FB001','110','R001','2026-03-10','10:00','12:00','Volleyball'),
('FB002','120','R002','2026-03-11','14:00','16:00','Yoga practice'),
('FB003','130','R001','2026-03-12','09:00','11:00','Basketball'),
('FB004','140','R003','2026-03-13','13:00','15:00','Dance practice');

INSERT INTO ClassBooking VALUES
('CB001','110','CLS001','2026-03-05','N'),
('CB002','110','CLS002','2026-03-05','N'),
('CB003','120','CLS003','2026-03-06','N'),
('CB004','130','CLS001','2026-03-06','N'),
('CB005','130','CLS004','2026-03-07','N'),
('CB006','130','CLS002','2026-03-07','Y'),
('CB007','130','CLS001','2026-03-08','N'),
('CB008','130','CLS002','2026-03-08','N'),
('CB009','130','CLS003','2026-03-09','Y');


SELECT MemberID, FirstName, Surname, Telephone, Email
FROM Member
ORDER BY Surname;

```

```sql

SELECT FacilityBooking.BookingID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Room.RoomName,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       FacilityBooking.EndTime,
       FacilityBooking.ActivityDescription
FROM FacilityBooking
JOIN Member ON FacilityBooking.MemberID = Member.MemberID
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE MONTH(FacilityBooking.BookingDate) = MONTH(CURDATE())
AND YEAR(FacilityBooking.BookingDate) = YEAR(CURDATE())
ORDER BY FacilityBooking.BookingDate, FacilityBooking.StartTime;


```

```sql
SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       Room.RoomName,
       Class.MaxClassSize,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Class.StaffID = 'S01'
ORDER BY Class.DayOfWeek, Class.StartTime;


SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Room.RoomID = 'R002'
ORDER BY Class.DayOfWeek, Class.StartTime;
```

```sql
SELECT 'Facility Booking' AS BookingType,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       Room.RoomName,
       FacilityBooking.ActivityDescription AS Detail
FROM FacilityBooking
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE FacilityBooking.MemberID = '110'
UNION ALL
SELECT 'Class Booking' AS BookingType,
       ClassBooking.BookingDate,
       Class.StartTime,
       Room.RoomName,
       Class.ClassName AS Detail
FROM ClassBooking
JOIN Class ON ClassBooking.ClassCode = Class.ClassCode
JOIN Room ON Class.RoomID = Room.RoomID
WHERE ClassBooking.MemberID = '110'
ORDER BY BookingDate, StartTime;
```

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
ORDER BY TotalClassBookings DESC;
```

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       IFNULL(FacilityCount.Total, 0) AS FacilityBookings,
       IFNULL(ClassCount.Total, 0) AS ClassBookings,
       IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) AS TotalActivity
FROM Member
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM FacilityBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) FacilityCount ON Member.MemberID = FacilityCount.MemberID
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM ClassBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) ClassCount ON Member.MemberID = ClassCount.MemberID
WHERE IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) > 0
ORDER BY TotalActivity DESC;
```

fitness*

Limit to 1000 rows

```sql

SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings,
       (COUNT(*) - 5) * 500 AS ExtraChargeRs
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
HAVING COUNT(*) > 5
ORDER BY TotalClassBookings DESC;


SELECT Room.RoomID,
       Room.RoomName,
       Room.MaxCapacity,
       COUNT(FacilityBooking.BookingID) AS TotalBookings
FROM Room
LEFT JOIN FacilityBooking ON Room.RoomID = FacilityBooking.RoomID
GROUP BY Room.RoomID, Room.RoomName, Room.MaxCapacity
ORDER BY TotalBookings DESC;

```

fitness*

Limit to 1000 rows

```sql

SELECT DISTINCT Member.MemberID,
       Member.Surname,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Member.Email
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
WHERE ClassBooking.ExtraCharge = 'Y'
ORDER BY Member.Surname;


```